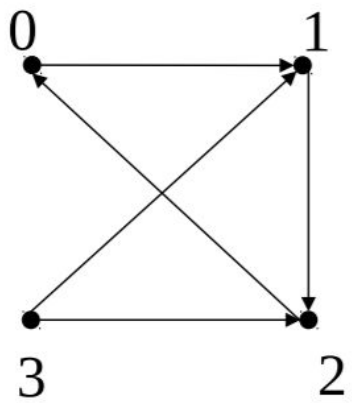
Graphs, exercises

1. Create a data structure for representing a digraph (digraf, graf orientat)/(oriented graph).

a. Using a matrix, matrice de adiacenţă/adjacency matrices.

Example is the one from the course:

	0	1	2	3
0	0	1	0	0
1	0	0	1	0
2	1	0	0	0
3	0	1	1	0



Create the digraph with a matrix.

1. Read the number of nodes, for example “n”; and m if we need
2. Create the matrix
3. Initialize with 0 or NULL
4. Read the connections.

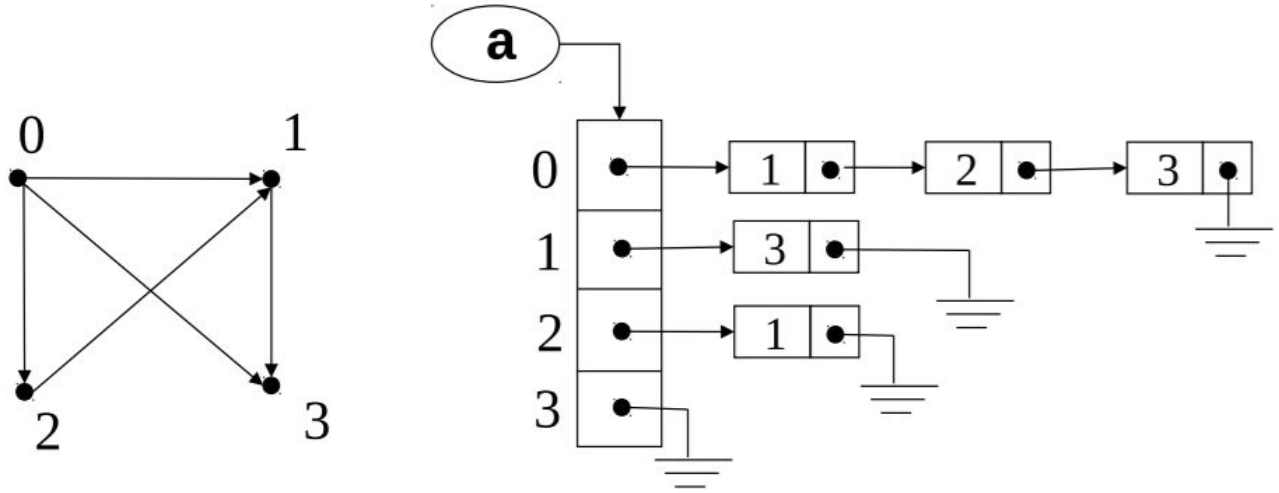
```
function create_adjacency_matrix()
begin
        n = 0
        read n

        matrix[0,n][0,n]
        for i=0 to n-1 do
                for j=0 to n-1 do
                        matrix[i][j] = 0 //we initialize the matrix

        for i=0 to n-1 do
                for j=0 to n-1 do
                        print “put 1 if there is a connection between node” + i + “ and node “ + j
                        read connection
                        if connection == 1 // we are interested if a connection is 1, else it will be 0
                                matrix[i][j] = 1
        return matrix
end
```

OR:

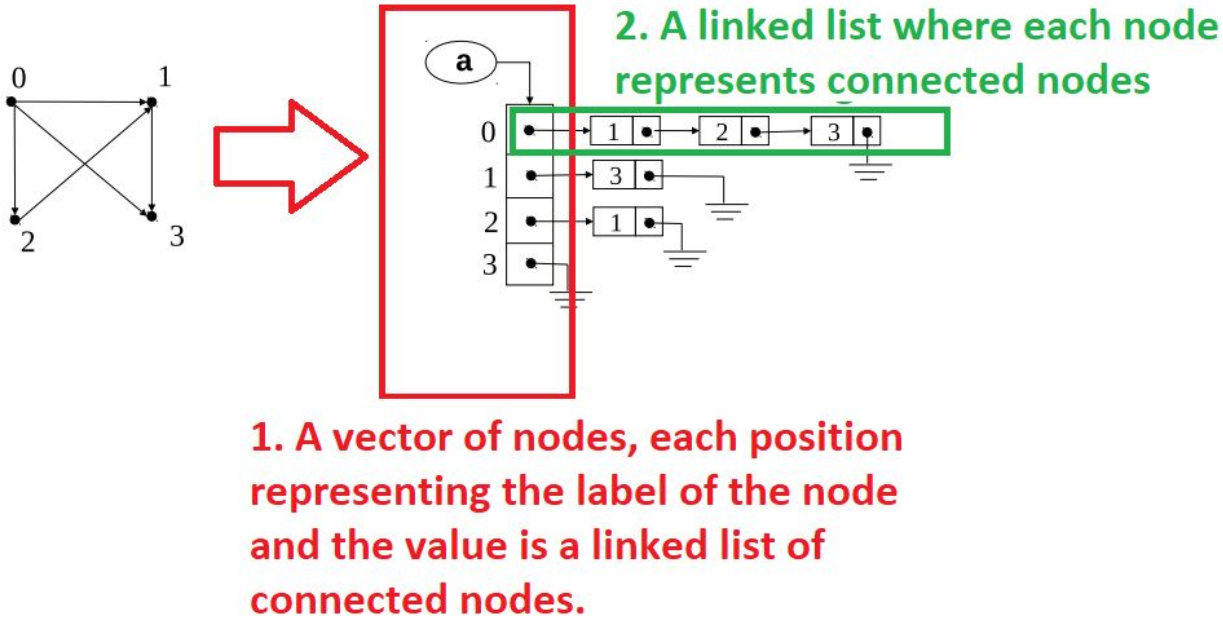
```
fuction MatrAd()
begin
1.    read n,nrmuchii
2.    for i<-0,n-1 do
3.         for j<-0,n-1 do
4.             M[i][j]=0
5.    For i<-0,nrmuchii-1 do
6.             Read a,b
7.             M[a][b]=1
8.    return M
end
```

b. **Using adjacency lists/liste de adiacenţă**

Example is the one from the course:



How do we do it? → We must define a structure that is a vector of nodes representing nodes from where the arc starts and each node should have a linked list of all connecting nodes (where the arc ends).
We basically have 2 components:



1. The vector of notes:

```
struct Digraph{
        int n; // the total number of nodes;
        Node* nodes[0,n]; // an array of start nodes that contain a linked list of connected nodes
}
```

2. The node structure

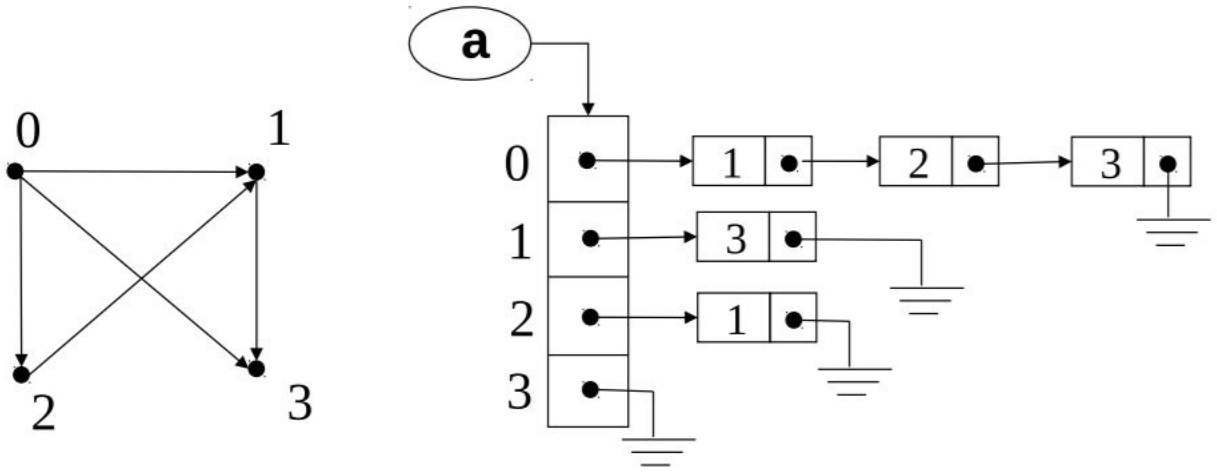
```
struct Node{
        int nodeNumber;
```

```
        Node* succ;
}

function create_adjacency_list():
begin
        Digraph digraph = new Digraph()
        read n
        digraph.n = n

        for i=0 to n-1 do
                digraph.nodes[i] = new Node() // we create n nodes

        read m // we suppose the user will give us how many arcs we have

        for i=0 to m-1
                read x,y // this is an arc from x→ y
                insert_arc(digraph, x, y)
        end
        return digraph
end


procedure insert_arc(digraph, x, y):
begin
        p = digraph.nodes[x] // we take the x-th

        while p→ succ != null && p→ nodeNumber != y do
                // if we are not at the end and there isn't a connection already, continue
                p = p → succ
        end

        // we are now either at the last node or at a node that has y
        // we will see if the node has y, in which case we do nothing or create a new node
        if p→ nodeNumber != y then
                q = new Node()
                q → nodeNumber = y
                q → succ = NULL
                p→ succ = q
        end
end
```

2. Given a graph D as an adjacency list, calculate the outdegree (grad exterior) of a node v.

Outdegree of a node = the number of arcs that exit the node

Example:



For 0 the outdegree is 3, because it has 3 arches to 1, 2 and 3.

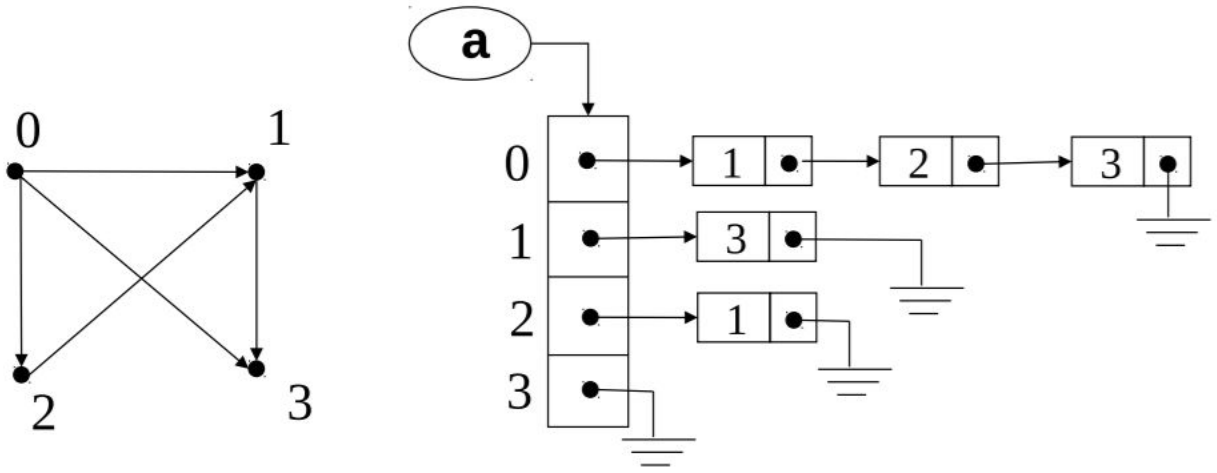
```
function extgrad(D, v)
begin
        p = d.nodes[v]
        ...
```

```
// D - digraph and x is the node no.
function outdegree(D,x):
begin
        x_node = D.nodes[x] // our node is in the node list on x position

        degree = 0

        while x_node != null do
                degree = degree ++
                x_node = x_node → succ
        end

        return degree
end
```

3. Given a graph D as an adjacency list, calculate the indegree (grad interior) of a node v.

Indegree of a node = the number of arcs that enter the node

Example:



For 1 the indegree is 2, because it has 2 arches from 0 and 2

We can also see that on the adjacency list 1 o is on 0 and 2, thus we have to search everywhere.

```
function int(D,v)
begin
        ...
```

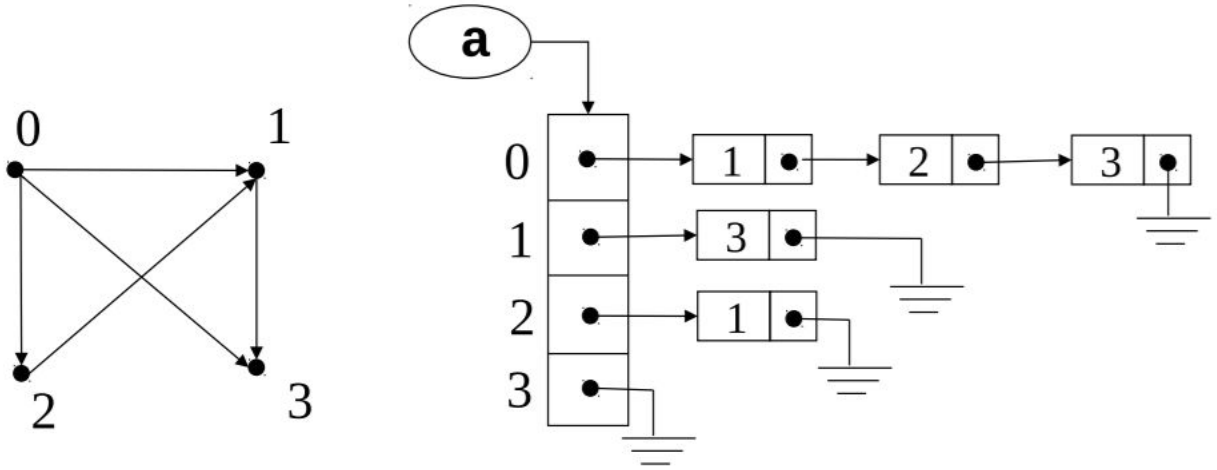
```
function indegree(D, x):
begin
        indegree = 0

        for i=0 to digraph.n - 1 do
                node = D.nodes[i] // take the node
                while node != null do
                        if node → nodeNumber == x then
                                indegree = indegree ++
                                break // we cannot have another node that is the same
                        node = node → succ
                end
        return indegree
end
```

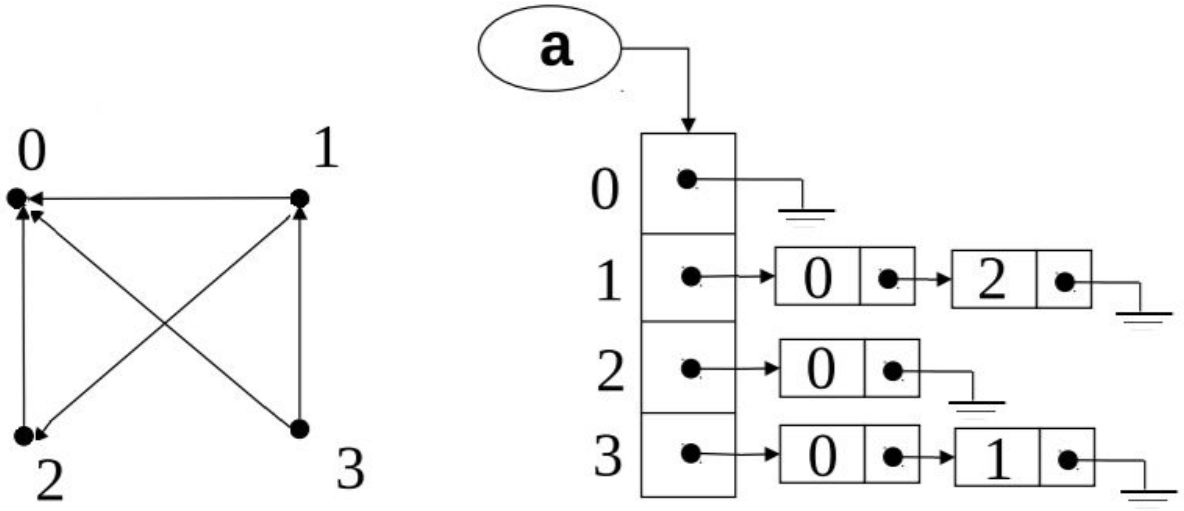
4. For a given digraph D, create and return the transposed digraph; a transposed digraph is obtained by inverting the direction of every arc in the digraph.

Pentru un digraf D, construiește D transpus, și anume graful obtinut prin inversarea sensurilor arcelor lui D.

Initial node:

After the inversion





```
procedure insert_arc(digraph, x, y):
begin
        p = digraph.nodes[x] // we take the x-th

        while p→ succ != null && p→ nodeNumber != y do
                // if we are not at the end and there isn't a connection already, continue
                p = p → succ
        end

        // we are now either at the last node or at a node that has y
        // we will see if the node has y, in which case we do nothing or create a new node
        if p→ nodeNumber != y then
                q = new Node()
                q → nodeNumber = y
                q → succ = NULL
                p→ succ = q
        end
end

function invert_digraph(D):
begin
        Digraph inverted // the inverted graph
        inverted.n = d.n // we will copy the number of nodes

        //for pentru new node()

        for i=0 to n-1 do
                node = d.nodes[i] // take the current node
                while node != null do
                        insert_arc(inverted, node->nodeNumber, i)
                        //insert in inverted graph an arc between our current note and the "parent"
                        node = node → succ
                end
        return inverted
end
```